

Diagnosis Before Recovery: Turning Agent Failures into Selective Self-Correction

Pan Wang Yihao Hu Hang Wang Zirui Lv Xin Zhang Jianshe Li Jiang-Ming Yang
Wei Wu Yongqi Tong✉

Ant International

Contact: Pan Wang diogenescask@gmail.com

✉ Corresponding Author: yqitong@gmail.com

Abstract

Self-correction is particularly useful when a failure constrains the next repair. Coding agents benefit from this property because compilers, tests, and execution traces turn many failures into typed recovery signals, but broad language-agent tasks often expose only a coarse task failure. This creates a tension for generic recovery playbooks: they broaden the agent’s context precisely when the system needs a narrower repair interface, mixing incompatible signals for invalid actions, missing procedures, and strict-format errors. Our insight is that development-set failures can recover part of the missing diagnostic substrate by deciding which recovery interventions are admissible before test-time correction. We propose DARC, a diagnosis-guided recovery harness that profiles task-family failure modes, prunes mismatched interventions from a shared recovery library, and freezes a verifier-selected success-cost policy for deployment. This causal order makes correction selective: the harness first determines what kind of failure can be repaired, then decides how much recovery evidence to spend. In ALFWorld, AppWorld, and XBRL Finance, the same protocol yields an action-validity harness, a procedural-recovery fallback, and a format-precision retrieval policy; in each evaluated setting it improves average task performance over base agents and broad playbooks while reducing environment steps or retrieval budget. Our experiments show that failures need not trigger uniformly more context: DARC turns self-correction from prompt expansion into recovery-interface design. DARC provides a practical route toward more reliable agents in domains where compiler-like feedback is absent: making failures actionable before making contexts larger.

Keywords: language agents, self-correction, failure diagnosis, recovery policies, agent evaluation

1 Introduction

Coding agents have become one of the most visible successes of language-agent research. A major reason is not only that code models improved, but that software environments provide an unusually powerful recovery interface: compilers expose syntax and type errors, tests expose behavioral mismatches, and execution traces localize intermediate states [13, 25, 37, 88, 95]. These signals do more than add information. They make failure actionable. A type error, a failing assertion, and a timeout call for different repairs; the environment helps the agent distinguish them.

Most agent tasks are asked to self-correct without this diagnostic substrate. An embodied household agent may fail after issuing an invalid action, but the benchmark may only report that the episode was not solved. A multi-application assistant may miss a goal because it lacks the right API workflow, while a financial extraction agent may locate the evidence but violate a strict schema. These are not the same failure. Yet many recovery recipes treat them as if they were:

append reflection, retrieve more examples, expose a longer tool manual, or run a broad recovery playbook. That strategy is attractive because it is generic, but it suffers from three concrete problems. **❶ Intervention Mismatch:** a recovery signal chosen without diagnosis often cannot address the failure at hand, such as exposing an exhaustive API manual to fix an invalid embodied action or a strict-format violation. **❷ Recovery Interference:** irrelevant procedures can pull an embodied agent away from state-compatible actions, long manuals can mix incompatible workflow rules, and demonstrations for one failure mode can degrade another. **❸ Uncontrolled Recovery Cost:** triggering every recovery signal on every failure inflates environment steps, retrieval budget, and inference tokens regardless of whether the extra evidence is needed.

The key question is therefore not whether agents need more recovery material, but *which failure should be allowed to trigger which recovery signal*. This paper studies *diagnosis-guided agent self-correction*: using development failures

to recover part of the compiler-like role that broad agent tasks lack. Instead of treating a failure as a request for more context, we treat it as evidence about the dominant bottleneck of a task family. The output is a *recovery harness*: a frozen policy that links a diagnosed failure mode to a restricted set of admissible recovery interventions, such as an action-validity guard, a procedural API source, a local induction rule, or a retrieval-demonstration budget.

We propose **DARC**, a framework for **D**iagnostics-guided **A**gent **R**ecovery and **C**orrection. DARC makes two design choices explicit. **First**, to address ❶, it profiles development-set failures and restricts the recovery library before test evaluation, so interventions that cannot address the diagnosed failure mode are excluded rather than mixed into a generic playbook. **Second**, to resolve ❷ and ❸, within the admissible set it uses training-set verifier feedback to distill a short success-cost policy for deployment, so that expensive recovery evidence is spent only when cheaper interventions fail. To ensure stability and auditability, we freeze this diagnosis at the task-family level rather than learning it online.

We evaluate this protocol in three settings chosen to stress different recovery needs. ALFWorld tests action validity, where recovery should constrain state-incompatible actions rather than expose unrelated manuals. AppWorld tests procedural breadth, where recovery should reveal richer API workflow knowledge only when cheaper procedural sources fail. XBRL Finance tests format precision, where recovery should tune demonstration evidence without expanding into a broad playbook. These settings allow us to evaluate whether failure diagnosis makes self-correction more reliable and cost-aware across diverse bottlenecks.

Our contributions are:

- We formulate diagnosis-guided agent self-correction as the problem of turning ambiguous task failures into actionable recovery signals, motivated by the compiler-like feedback available in coding but often absent in broad agent tasks.
- We propose DARC, which constructs recovery harnesses by development-set failure diagnosis, admissible-intervention restriction, and success-cost policy distillation.
- We evaluate DARC on ALFWorld, AppWorld, and XBRL Finance, showing that targeted recovery improves average success or accuracy under the reported protocols while reducing environment steps or retrieval budget.
- We provide extensive ablations isolating the impact of correct, generic, and mismatched recovery policies, demonstrating that diagnosis-guided restriction is essential for efficient self-correction.

2 Related Work

Self-correction from execution feedback. Recent work shows that agents can improve when feedback is structured enough to guide repair. In code generation, Self-

Debugging uses execution results and explanations to refine programs [13], while LDB segments programs into execution units and verifies intermediate runtime states [95]. Related code-agent and program-synthesis work uses generated tests, execution traces, benchmarks, or agent-computer interfaces to expose concrete repair signals [6, 10, 12, 25, 47, 80, 85, 88]. Work on learning from mistakes and negative examples similarly suggests that errors can be useful supervision when their structure is exposed [21, 71, 77, 78]. Voyager uses environment feedback, execution errors, and self-verification to grow a skill library in Minecraft [75]. DARC takes this compiler-like feedback as motivation, but targets settings where such feedback is not naturally available: it uses development failures to build a task-family recovery interface for non-coding agent tasks.

Language-agent recovery and reflection. ReAct [90] interleaves reasoning and acting, Reflexion [64] stores verbal feedback, and recursive critique [30] uses feedback loops for computer-control tasks. Broader reasoning and refinement methods use chain-of-thought, self-consistency, decomposition, search, or process supervision to make intermediate reasoning states more explicit [14, 31, 40, 46, 70, 72, 76, 81–83, 91, 93, 96]. These methods demonstrate the value of self-correction, but the recovery mechanism is often applied uniformly across failures. DARC instead makes the recovery policy conditional on a diagnosed failure mode: action-validity recovery, procedural recovery, and format-precision recovery expose different interventions.

Prompt optimization and generic context. Frameworks such as DSPy [28] and teleprompter-based optimization [48] tune instructions and demonstrations over validation data. Tool and multi-agent systems expose external modules, APIs, planning roles, or conversation protocols to language models [19, 26, 34, 36, 52, 54, 56, 60, 62, 67, 84]. Other lines use reinforcement learning or preference optimization to update model behavior [4, 8, 32, 38, 49, 51, 59, 61, 79, 86], including multi-turn or device-control agents [7, 55, 97]. These approaches are complementary to DARC. Our focus is not weight updates or arbitrary prompt search, but the construction of a recovery interface that decides which feedback and interventions are relevant for a diagnosed failure mode.

When more context is not neutral. The motivation for diagnosis-guided recovery is strengthened by evidence that irrelevant or poorly placed context can hurt language-model behavior. GSM-IC shows that irrelevant context can distract LLM reasoning [63], while Lost in the Middle shows that models do not robustly use long contexts even when relevant information is present [42]. Long-context, real-world chat, format-following, and safety studies similarly show that performance depends on where information appears, whether irrelevant material is present, and whether the task constraints are precise [9, 20, 35, 41, 50, 69, 89]. In agent settings, a generic playbook can similarly mix unrelated pro-

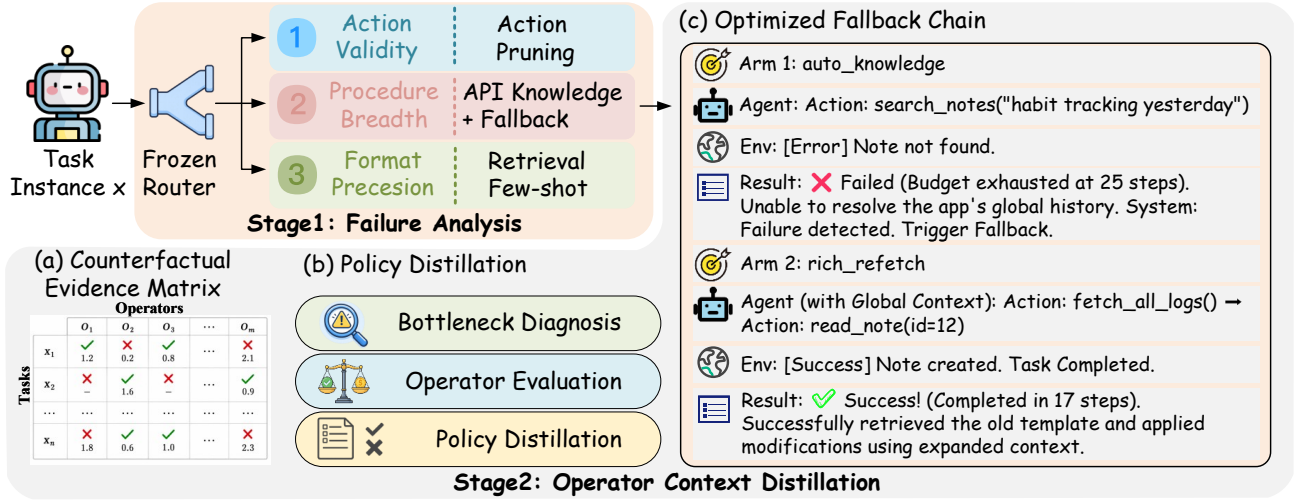


Figure 1: DARC constructs a diagnosis-guided recovery harness. Development failures are converted into a task-family failure diagnosis, which restricts the candidate recovery interventions. Training-set verifier feedback then scores candidate fallback policies, and the selected policy is frozen before test evaluation.

cedures, constraints, and demonstrations. DARC addresses this by restricting recovery context before policy search rather than appending a universal context bundle after every failure.

Cascades, routing, and adaptive computation. LLM cascade methods such as FrugalGPT [11] and AutoMix [2] route queries among models to trade cost for accuracy. Retrieval-augmented and adaptive-retrieval methods decide what evidence to fetch or how much retrieval is needed for a query [5, 16, 17, 22–24, 27, 33, 87]. DARC routes among *recovery interventions* rather than base models. Scaling or switching the model does not necessarily fix an invalid action, a missing API procedure, or an output-format violation. The closest matched baseline is therefore a validation-selected full-library recovery cascade without diagnosis; we report this controlled comparison on ALFWorld in Section 6 (Table 14), where diagnosis matches full-library accuracy while searching a much smaller policy space.

3 Problem Setup: Diagnosis-Guided Recovery

Let $\mathcal{X}$ denote a task family and let an agent produce an output or trajectory y for each task $x \in \mathcal{X}$. Evaluation is given by a task verifier $s(x, y) \in \{0, 1\}$ and an execution cost $c(x, y) \geq 0$, such as environment steps, latency, or retrieval budget. We assume a library of candidate recovery interventions $\mathcal{R}$, where an intervention may be an action guard, an API procedure source, a local induction rule, or a few-shot retrieval budget. We use *intervention* for an executable correction signal that can be attached to a base agent and evaluated by the task verifier. A failure signal is *actionable* when it selects or parameterizes an admissible intervention rather than merely adding more undifferentiated

context.

The central object in this paper is a *recovery harness*. A harness for failure mode m is defined as a tuple

$$H_m = (m, \mathcal{R}_m, \pi_m), \quad (1)$$

where $\mathcal{R}_m \subseteq \mathcal{R}$ is the set of interventions admissible for that diagnosed failure mode and $\pi_m = (r_1, \dots, r_L)$ is the ordered deployment policy over those interventions. In our instantiation, m is assigned at the task-family level from development-set failure profiles. Each benchmark family or split pair defines one $\mathcal{X}$ and one fixed recovery policy before test evaluation.

The recovery objective is to select a short policy that improves task success without invoking unnecessary or mismatched interventions:

$$\max_{\pi \in \Sigma(\mathcal{R}_m)} \mathbb{E}_{x \sim \mathcal{X}} [s(x, \pi)] - \lambda \max(0, \mathbb{E}_{x \sim \mathcal{X}} [c(x, \pi)] - \tau_{\text{free}}), \quad (2)$$

where $\Sigma(\mathcal{R}_m)$ is the space of bounded-length recovery policies, λ controls the cost-success tradeoff, and τ_{free} is the cost allowance of a standard attempt. This formulation makes two questions explicit: which recovery signals are actionable for this failure mode, and in what order should they be invoked?

4 DARC: Diagnosis-Guided Agent Recovery and Correction

DARC constructs a recovery harness in two stages (Figure 1). The diagnosis stage turns development failures into a task-family failure mode and restricts the recovery interventions that may be used. The policy-distillation stage freezes a cost-aware fallback policy over that restricted intervention set. Algorithm 1 summarizes the procedure.

Algorithm 1 DARC Recovery-Harness Construction

Require: Task family $\mathcal{X}$, candidate intervention library $\mathcal{R}$, training tasks $\mathcal{D}_{\text{train}}$, failure-profile data $\mathcal{D}_{\text{dev}}$, penalty λ .

- 1: **Profile:** Run the base agent on $\mathcal{D}_{\text{dev}}$ and measure failure signatures.
- 2: **Diagnose:** Select dominant failure mode m and admissible intervention set $\mathcal{R}_m \subseteq \mathcal{R}$.
- 3: **Evaluate interventions:** For each $x_i \in \mathcal{D}_{\text{train}}$ and intervention $r_j \in \mathcal{R}_m$, record success $s(x_i, r_j)$ and cost $c(x_i, r_j)$.
- 4: **Distill policy:** Enumerate bounded candidate policies $\pi \in \Sigma(\mathcal{R}_m)$ and score them by $J(\pi)$.
- 5: **Freeze:** Deploy $\pi^* = \arg \max_{\pi} J(\pi)$ as the task-family recovery policy.

Candidate intervention library. The candidate library $\mathcal{R}$ is fixed before test evaluation. It contains the recovery interventions available to all relevant baselines in a benchmark family: action guards for ALFWorld, procedural knowledge sources and local induction rules for AppWorld, and retrieval-demonstration budgets for Finance. DARC does not invent new interventions at test time; it restricts this pre-specified library to the interventions that match the diagnosed failure mode.

4.1 Failure Diagnosis

DARC converts development-set failure analysis into a harness restriction. For each task family, we execute the base agent on development tasks and measure observable failure signatures: invalid or state-incompatible actions in ALFWorld, missing cross-application procedures in AppWorld, and exact-format violations in XBRL Finance. These signatures identify a dominant failure mode and determine the admissible intervention set $\mathcal{R}_m$.

This frozen, task-family-level diagnosis makes the harness auditable, avoids test-time updates, and prevents the policy from observing test labels. While the framework can support learned per-instance routing, our experiments evaluate the controlled setting where each family instantiates one dominant failure mode. Appendix C lists the failure signals and intervention sets, while Appendix D provides representative development-trace diagnosis examples.

The output of diagnosis is not a longer prompt; it is a constrained recovery space. ALFWorld receives an action-validity harness, AppWorld receives a procedural-knowledge harness, and Finance receives a format-precision harness based on retrieval demonstrations. This restriction is the key difference from a universal playbook: interventions that cannot address the diagnosed failure mode are excluded before policy search, reducing irrelevant or conflicting recovery context.

4.2 Recovery-Policy Distillation

Given $\mathcal{R}_m$, DARC estimates which intervention or intervention sequence should be used at deployment time. For each training task x_i and intervention r_j , we collect

$$E_{i,j} = (s(x_i, r_j), c(x_i, r_j)), \quad (3)$$

where success is measured by the same task verifier used for training-set adaptation and cost is measured in the relevant budget unit. This uses training-set success feedback; it does not use test labels. Section 5 reports this accounting explicitly for all methods.

DARC evaluates a candidate policy $\pi = (r_1, \dots, r_L)$ as a short-circuit policy: the agent tries interventions in order and stops when one succeeds. The halting index is

$$h_i^*(\pi) = \begin{cases} \min\{t : s(x_i, r_t) = 1\}, & \text{if any intervention in } \pi \text{ succeeds,} \\ L, & \text{otherwise.} \end{cases} \quad (4)$$

The empirical success and cost are

$$\widehat{\text{succ}}(\pi) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_i \max_{r_t \in \pi} s(x_i, r_t), \quad (5)$$

$$\widehat{\text{cost}}(\pi) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_i \sum_{t=1}^{h_i^*(\pi)} c(x_i, r_t). \quad (6)$$

The deployed policy maximizes

$$J(\pi) = \widehat{\text{succ}}(\pi) - \lambda \max(0, \widehat{\text{cost}}(\pi) - \tau_{\text{free}}). \quad (7)$$

The short-circuit estimate assumes that intervention attempts are evaluated under a controlled retry protocol: later interventions are invoked only after an earlier intervention fails, and the verifier can attribute success and cost to the invoked intervention. In resettable environments, each intervention can be evaluated from the same initial state; in retrieval and extraction settings, interventions differ only in the supplied evidence budget. This assumption is explicit because it determines when an offline evidence matrix is a valid proxy for deployment. Table 1 summarizes the deployment semantics used by each benchmark family.

4.3 Structural Properties

The success term in DARC is a coverage objective over recovery interventions. For a subset $A \subseteq \mathcal{R}_m$, define $f(A) = \frac{1}{n} \sum_i \max_{r \in A} s(x_i, r)$. Then f is monotone and submodular, because adding an intervention can only cover additional tasks and marginal gains shrink as the covered set grows. This explains why short policies can capture most of the reachable recovery set in our finite candidate spaces.

The full objective in Eq. 7 is an ordered, cost-penalized policy objective, so the coverage result alone does not establish optimal cost-aware ordering. Instead, diagnosis reduces the candidate space and policy distillation enumerates bounded

Table 1: Deployment semantics for the reported DARC policies. The policy is frozen before test evaluation; later interventions are invoked only after earlier interventions fail under the benchmark verifier.

Benchmark	Later intervention observes	Side-effect control	Reported cost
ALFWorld	Same episode specification with a stricter action-validity harness	Resettable evaluation from the same initial environment state	Environment steps, invalid actions, seconds per episode
AppWorld	Richer procedural context after a cheaper procedural source fails	Controlled task attempts with success attributed by the task verifier	Environment steps and task/s-scenario completion
Finance	Same extraction query with a different retrieval-demonstration budget	Stateless extraction; no environment side effects	Retrieval budget k and answer accuracy

policies inside that space. For a fixed finite policy class Σ_K , standard uniform convergence gives that the empirical maximizer approaches the best population policy at rate $O(\sqrt{\log |\Sigma_K|/n})$ under bounded per-task costs. Since $|\Sigma_K|$ decreases with $|\mathcal{R}_m|$, diagnosis-guided restriction supports selection within a fixed finite candidate class; transfer across new failure modes remains an empirical question.

5 Experiments

We evaluate whether diagnosis-guided recovery improves agent self-correction under controlled adaptation budgets. The experiments answer four questions:

- **RQ1 (Performance):** How does DARC compare with base agents and generic playbook baselines?
- **RQ2 (Transfer):** Do frozen recovery policies transfer across related splits or tasks?
- **RQ3 (Diagnosis):** How much does selecting the correct diagnosis-guided recovery policy matter relative to generic or mismatched policies?
- **RQ4 (Cost):** Does the distilled policy reduce unnecessary environment steps or retrieval budget?

5.1 Experimental Setup

Benchmarks and recovery diagnoses. We instantiate one dominant failure diagnosis per benchmark family (Appendix C). **ALFWorld** [65] is treated as an action-validity setting, so the recovery harness contains task-specific action guards. **AppWorld** [73] is treated as a procedural-knowledge setting, so the harness contains Auto-Knowledge, local induction, and retrieval fallback interventions. **XBRL Finance** [45, 74] is treated as a format-precision setting, so the harness restricts interventions to retrieval-based demonstrations and distills the retrieval budget k .

Baselines. We compare against **Base LLM**, **ICL** [1], **MIPROv2** [48], **GEPA** [3], and **ACE** [94]. These baselines cover many-shot in-context learning, instruction/demonstration optimization, and reflective prompt evolution [28]. All offline-adaptation methods use the same training split and adaptation budget available to DARC unless a baseline is not applicable to a benchmark. Appendix G summarizes the mechanisms.

Table 2: Information used during adaptation. “Train success” denotes training-set verifier or task-success feedback used to score candidate policies. “Answer labels” denotes direct use of ground-truth answers to construct prompts, demonstrations, or optimized instructions. No method uses test labels.

Method	Train success	Answer labels	Test labels
Base LLM	No	No	No
ICL	No	Yes	No
MIPROv2 / GEPA	Yes	Yes	No
ACE	Yes	No	No
DARC	Yes	No	No

Supervision and feedback accounting. DARC uses training-set verifier outcomes to estimate intervention success in the evidence matrix; it does not use test labels. This differs from prompt-optimization baselines that directly use answer labels to construct prompts or demonstrations. Table 2 separates these forms of information.

Evaluation scope. We report all quantitative metrics as empirical estimates across the respective evaluation splits, and we complement the headline comparison with scenario-cluster bootstrap confidence intervals and paired significance tests to assess whether the observed gaps exceed split-level sampling noise (Figure 2). To isolate the impact of our diagnosis-guided restriction, the ablation studies compare three recovery strategies: (1) the *correct* policy, which aligns interventions with the diagnosed failure mode; (2) a *generic* baseline that uses a standard, unconstrained recovery playbook; and (3) a *mismatched* policy that acts as a negative control by applying an incompatible intervention set. This design separates the actual benefits of targeted recovery from the simple effect of adding more context.

5.2 Main Benchmark Results

Tables 3 and 4 report the main results. Across the evaluated settings, DARC generally improves average performance over base agents and generic playbooks, but the gains are not uniform across every metric. In particular, ACE matches or exceeds DARC on some AppWorld challenge metrics for Qwen3.5-27B and Qwen3.6-27B [57, 58]. We therefore interpret these results as evidence for the average utility of diagnosis-guided recovery, though the gains vary by metric.

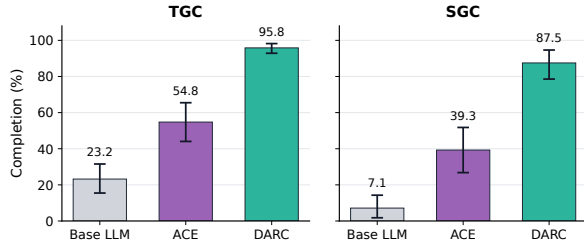


Figure 2: AppWorld Test-Normal completion for DeepSeek-V4-Flash with 95% scenario-cluster bootstrap confidence intervals (20,000 resamples over 56 scenarios). The intervals of DARC do not overlap those of ACE or the base agent on either TGC or SGC, and paired scenario-level tests with Holm correction confirm the gaps are significant.

On ALFWorld, the action-validity recovery policy substantially improves both seen and unseen splits. With DeepSeek-V4-Flash, valid-unseen success increases from 39.55% for the base agent and 54.48% for ACE to 90.30%. On AppWorld, the procedural recovery policy gives the strongest gains on Test-Normal and improves average TGC for all three base models, while challenge-split SGC remains competitive with ACE rather than uniformly better. On Finance, the format-precision policy reaches 94.50% macro accuracy with DeepSeek-V4-Flash, compared with 80.50% for ACE and 74.00% for MIPROv2 under the reported protocol. As detailed in Table 4, “Answer Labels” follows Table 2: whether ground-truth answers are directly used to construct prompts or demonstrations during adaptation. DARC uses training-set verifier outcomes to score retrieval budgets but does not use test labels.

Statistical significance. Because agent benchmarks are evaluated on finite task splits, we quantify the uncertainty of the headline AppWorld comparison at the scenario level. AppWorld Test-Normal contains 168 tasks nested within 56 scenarios (three tasks per scenario); since outcomes within the same scenario are correlated, we treat the scenario rather than the individual task as the independent statistical unit. We align all methods by task identifier, compute per-scenario TGC as the fraction of completed tasks and SGC as an indicator that all three tasks are completed, and estimate 95% confidence intervals with 20,000 scenario-cluster bootstrap resamples shared across methods. Figure 2 reports the resulting intervals for DeepSeek-V4-Flash: those of DARC do not overlap those of ACE or the base agent. Using a two-sided paired t -test for per-scenario TGC and an exact McNemar test for paired SGC outcomes, we confirm that all DARC-versus-baseline comparisons remain significant across the 56 scenarios after Holm correction. These intervals quantify scenario-sampling uncertainty rather than variation across model-training seeds.

5.3 Cross-Task Transferability

We next test whether a frozen recovery policy can transfer across related tasks. For Finance, we distill a retrieval-budget policy on Formula and deploy it on FiNER tags, and vice versa. For ALFWorld, we distill on valid_seen and deploy on valid_unseen. As shown in Table 5, the target-tuned row is shown only when a target-specific policy was available.

These within-family transfer results suggest that the frozen recovery policy can preserve much of the target-tuned performance when the failure mode remains similar. For Finance, the frozen retrieval-budget policy remains close to the target-tuned policy: Formula-to-FiNER reaches 89.50% versus 90.00%, and FiNER-to-Formula reaches 96.00% versus 99.00%. For ALFWorld, the same action-validity policy transfers from valid_seen to valid_unseen and reaches 90.30%.

6 Analysis and Ablation

The diagnosis-guided view predicts that recovery interventions are not interchangeable across failure modes. A retrieval demonstration can help Finance formatting but cannot make an ALFWorld action admissible. An action guard can prune embodied commands but cannot supply the AppWorld API procedure needed for a multi-application workflow. We test this prediction by comparing correct, generic, and mismatched recovery policies.

6.1 Ablation: Recovery Diagnosis

Table 6 shows that the correct recovery policy (aligned with the diagnosed failure mode) outperforms both a generic playbook and a mismatched intervention set across all evaluated settings. For AppWorld Test-Normal (evaluated under a stricter 25-step budget not comparable to Table 3), correct diagnosis reaches 70.24% TGC compared to 61.90% (generic) and 64.88% (mismatched), while achieving the best SGC with fewer environment steps than the generic playbook (36.60 vs. 42.97).

While AppWorld confidence intervals overlap, the directional advantage is consistent. Notably, the mismatched policy exceeds the generic playbook on AppWorld TGC. This supports our premise that excessive irrelevant context (generic) can be more harmful than largely inert mismatched interventions, though the expected ordering holds under the stricter SGC metric. The penalty for mismatched interventions is much larger on ALFWorld and Finance, where they actively conflict with the required recovery behavior.

Overall, these results highlight the necessity of diagnosis-guided restriction: adding context only helps when interventions match the underlying failure mode. Further ablations (Appendix H) confirm that isolating the diagnosis step maintains accuracy while significantly improving search efficiency and stability.

Table 3: Benchmark results on ALFWorld and AppWorld. For ALFWorld, we report success rates on valid_seen and valid_unseen. For AppWorld, TGC is Task Goal Completion and SGC is Scenario Goal Completion. Bold marks the best value within each base-model block and metric.

Method	ALFWorld			AppWorld					
	valid_seen↑	valid_unseen↑	Macro Avg.↑	Test-Normal		Test-Challenge		Average	
				TGC↑	SGC↑	TGC↑	SGC↑	TGC↑	SGC↑
DeepSeek-V4-Flash									
Base LLM	40.71%	39.55%	40.13%	23.21%	7.14%	13.43%	6.47%	18.32%	6.81%
ICL	46.43%	42.54%	44.48%	59.52%	42.86%	37.17%	19.42%	48.35%	31.14%
MIPROv2	54.29%	49.25%	51.77%	57.14%	35.71%	32.85%	10.07%	45.00%	22.89%
GEPA	59.29%	63.43%	61.36%	54.17%	30.36%	33.57%	12.95%	43.87%	21.65%
ACE	50.00%	54.48%	52.24%	54.76%	39.29%	35.73%	17.99%	45.25%	28.64%
DARC	93.57%	90.30%	91.94%	95.83%	87.50%	53.96%	30.22%	74.90%	58.86%
Qwen3.5-27B									
Base LLM	58.57%	55.22%	56.90%	42.26%	23.21%	23.74%	12.23%	33.00%	17.72%
ICL	72.14%	70.90%	71.52%	75.00%	57.14%	59.71%	38.13%	67.36%	47.64%
MIPROv2	69.29%	60.45%	64.87%	4.76%	0.00%	6.95%	0.72%	5.86%	0.36%
GEPA	79.29%	72.39%	75.84%	50.60%	17.86%	34.29%	12.95%	42.44%	15.40%
ACE	80.00%	81.34%	80.67%	79.17%	64.29%	74.58%	51.08%	76.88%	57.68%
DARC	94.29%	95.52%	94.90%	93.45%	89.29%	65.47%	41.01%	79.46%	65.15%
Qwen3.6-27B									
Base LLM	59.29%	56.72%	58.00%	42.86%	32.14%	26.14%	12.95%	34.50%	22.55%
ICL	75.00%	76.12%	75.56%	60.71%	46.43%	41.73%	25.18%	51.22%	35.80%
MIPROv2	63.57%	55.22%	59.40%	14.29%	8.93%	6.47%	2.88%	10.38%	5.91%
GEPA	90.00%	83.58%	86.79%	45.83%	30.36%	34.53%	18.71%	40.18%	24.53%
ACE	80.71%	78.36%	79.54%	67.86%	55.36%	53.00%	32.37%	60.43%	43.86%
DARC	97.14%	95.52%	96.33%	86.31%	78.57%	47.72%	32.37%	67.02%	55.47%

Table 4: Finance benchmark results on FiNER tags and Formula extraction.

Method	Answer Labels	FiNER / Tags Acc↑	Formula Acc↑	Average↑
DeepSeek-V4-Flash				
Base LLM	No	68.00%	8.00%	38.00%
ICL	Yes	77.50%	66.00%	71.75%
MIPROv2	Yes	78.50%	69.50%	74.00%
GEPa	Yes	82.50%	61.00%	71.75%
ACE	No	83.00%	78.00%	80.50%
DARC	No	90.00%	99.00%	94.50%
Qwen3.5-27B				
Base LLM	No	47.50%	24.00%	35.75%
ICL	Yes	69.50%	45.50%	57.50%
MIPROv2	Yes	72.00%	58.00%	65.00%
GEPa	Yes	68.00%	61.00%	64.50%
ACE	No	82.00%	79.00%	80.50%
DARC	No	87.50%	98.50%	93.00%
Qwen3.6-27B				
Base LLM	No	53.00%	36.00%	44.50%
ICL	Yes	67.00%	46.00%	56.50%
MIPROv2	Yes	72.50%	60.00%	66.25%
GEPa	Yes	77.50%	60.00%	68.75%
ACE	No	81.50%	75.00%	78.25%
DARC	No	86.50%	98.50%	92.50%

6.2 Matched-Information Fairness

The large ALFWorld gain invites a fairness concern: does DARC win only because its action-validity guard sees privileged admissibility information that baselines do not? We test this directly on the full 134-task valid_unseen split, with all methods using the same DeepSeek-V4-Flash backbone, identical environment observations, the full set of admissible actions returned by the environment, a 50-step budget, and the same evaluation script. Crucially, DARC’s

Table 5: Cross-task transferability of frozen DARC policies. Finance transfers between the Formula and FiNER-tags tasks; ALFWorld transfers from valid_seen to valid_unseen.

Method	Finance (Acc↑)		ALFWorld (Succ↑)
	Formula → FiNER	FiNER → Formula	seen → unseen
Base LLM	68.00%	8.00%	39.55%
ICL	77.50%	66.00%	42.54%
MIPROv2	78.50%	69.50%	49.25%
GEPa	82.50%	61.00%	63.43%
ACE	83.00%	78.00%	54.48%
DARC (Frozen)	89.50%	96.00%	90.30%
DARC (Target-Tuned)	90.00%	99.00%	—

Table 6: Impact of selecting the correct recovery diagnosis.

Benchmark	Metric	Correct	Generic	Mismatched
ALFWorld	Success↑	91.94%	52.24%	52.24%
AppWorld	TGC↑	70.24%	61.90%	64.88%
AppWorld	SGC↑	53.57%	50.00%	46.43%
Finance	Macro↑	94.50%	80.50%	37.75%

guard is not a model-probability ranker and does not read the environment’s ground-truth action: it is a deterministic, diagnosis-aware rule scorer that ranks each admissible command a by

$$S(a) = S_{\text{goal}} + S_{\text{phase}} + S_{\text{search}} + S_{\text{train-prior}} - S_{\text{conflict}} - S_{\text{loop}}, \quad (8)$$

and keeps the top 12. All terms are computed from the task description and training-set priors; none use test labels or environment-provided correct actions.

Table 7 decomposes the sources of the gain. In this table, “Base + matched metadata” adds only parsed task type,

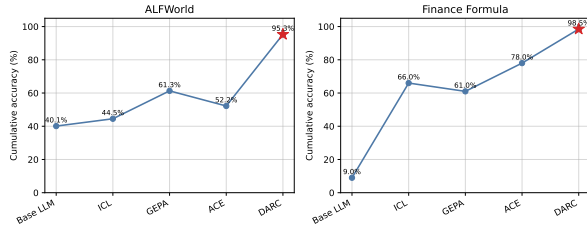


Figure 3: Cumulative solved episodes on ALFWorld as the maximum environment-step budget increases. DARC solves more episodes at lower budgets because the action-validity recovery policy reduces invalid or inefficient actions.

target object, and movable/goal containers on top of the full action set, without diagnosis, ranking, or direct execution. Numbers are a same-day matched rerun under the 50-step budget; the ranked-recovery row is the run reported as DARC valid_unseen in Table 3 (90.30%). They are not directly comparable to the valid_seen-selected frozen cascade in Table 14 (99.25%, different selection protocol). Granting the base agent the same parsed metadata (task type, target object, movable and goal containers) improves success only marginally (39.55%→43.28%), so information access alone is not the driver. Restricting the action view to a *random* top-12 collapses ACE (54.48%→25.37%), so action-space restriction by itself is not a free lunch and can hurt. Diagnosis as a bare label, without the ranked recovery operator, stays at base level (38.81%). The gain materializes only with the ranked recovery operator, which reaches 89–90% while cutting invalid actions per episode to 0.11. The ALFWorld improvement is therefore attributable to the quality of the diagnosis-guided *ranking* of already-available admissible actions, not to privileged information or restriction alone.

Appendix F isolates this further with a 2×2 factorial over the two components of the harness. Neither component works alone: the recovery prompt on the full action set is worth -0.75 pp, the ranked action view without the recovery prompt is worth $+3.73$ pp, and forcing the guard’s top-1 command with no language model in the loop reaches only 40.33%. The two components together are worth $+49.26$ pp, an interaction of $+46.27$ pp. The ALFWorld gain is thus a property of the recovery *interface*, where restriction and instruction are explicitly matched to each other, rather than of either prompt content or action pruning in isolation.

Table 7: Matched-information fairness evaluation on the ALFWorld valid_unseen split.

Method	Action view	Success $\uparrow$	Steps $\downarrow$	Invalid/ep $\downarrow$	vs. ACE
Base LLM	Full	39.55%	35.25	1.709	-14.93
Base + matched metadata	Full	43.28%	34.02	1.560	-11.19
ACE	Full	54.48%	30.06	1.784	0.00
ACE + random top- k	Random top-12	25.37%	42.16	7.403	-29.10
DARC, diagnosis only	Full	38.81%	34.69	2.507	-15.67
DARC, ranked recovery	Ranked top-12	90.30%	16.50	0.575	+35.82
DARC, full guard	Ranked top-12	89.55%	18.38	0.112	+35.07

6.3 Weight-Space Training Extension

Beyond test-time recovery, we conduct a preliminary probe into extending DARC to weight-space training. Using Qwen3-8B, we compare two full-parameter fine-tuning methods guided by a DARC-derived two-stage curriculum (diverse_replay then uniform_rollout, allocating 5.0M and 2.5M tokens respectively). **DARC-GRPO** updates weights using standard GRPO, while **DARC-OPSD** adds an online exponential-moving-average (EMA) teacher for policy self-distillation. Table 8 summarizes the results for the final checkpoints.

Table 8: Full evaluation of weight-space training extensions on ALFWorld. Models are fine-tuned from Qwen3-8B using a DARC-derived two-stage curriculum.

Method	valid_seen	valid_unseen	Macro $\uparrow$	Δ Base
Base Qwen3-8B	7.14%	4.48%	5.81%	–
DARC-GRPO	12.14%	8.96%	10.55%	+4.74 pp
DARC-OPSD	9.29%	6.72%	8.00%	+2.19 pp

Both weight-space variants improve success over the base model, suggesting that the DARC-derived curriculum transfers to weight-space training. DARC-GRPO improves macro success by 4.74 percentage points, while DARC-OPSD yields a smaller 2.19 pp gain, indicating that online self-distillation does not provide additional benefit over standard GRPO in this setting. Because these are single-seed runs with modest absolute gains, we treat this as a proof of concept rather than a fully verified training recipe.

6.4 Benchmark-Specific Failure Modes

Action validity in ALFWorld. The ALFWorld recovery policy restricts the agent to state-compatible actions. With DeepSeek-V4-Flash, DARC reduces invalid actions per episode to 0.091 (Table 10) and improves macro success to 91.94% (Table 3). This supports the interpretation that action validity is the dominant failure mode in this setting.

Procedure breadth in AppWorld. AppWorld requires multi-step API knowledge. The procedural recovery policy uses a fallback chain because the interventions are complementary: a local induction rule can recover tasks that an Auto-Knowledge source misses. The gains are strongest on Test-Normal; the Test-Challenge split remains harder and includes metrics where ACE is competitive or better.

Format precision in Finance. Finance shows that a recovery policy may tune a single intervention dimension rather than switch among heterogeneous interventions. Once the admissible set is restricted to retrieval demonstrations, DARC selects how much evidence to retrieve. Table 9 shows that larger retrieval budgets are not monotonic: $k = 8$ and $k = 16$ slightly reduce tag accuracy relative to smaller budgets. The distilled policy uses $k = 2$ for Formula and $k = 1$ for Tags, staying in the top macro-accuracy band while reducing the mean retrieval budget to 1.5 demonstrations.

Table 9: Finance retrieval-budget distillation. The DARC policy selects $k = 2$ for Formula and $k = 1$ for Tags, staying in the top macro-accuracy band with a mean budget of 1.5 demonstrations.

Method	Answer Labels	Formula	Tags	Macro	Mean k
Base LLM	No	8.0%	68.0%	38.00%	—
ICL	Yes	66.0%	77.5%	71.75%	4.0
MIPROv2	Yes	69.5%	78.5%	74.00%	—
GEPA	Yes	61.0%	82.5%	71.75%	—
ACE	No	78.0%	83.0%	80.50%	—
Fixed retrieval $k = 1$	No	96.0%	90.0%	93.00%	1.0
Fixed retrieval $k = 2$	No	99.0%	89.5%	94.25%	2.0
Fixed retrieval $k = 4$	No	99.0%	89.5%	94.25%	4.0
Fixed retrieval $k = 8$	No	99.0%	89.0%	94.00%	8.0
Fixed retrieval $k = 16$	No	99.0%	88.0%	93.50%	16.0
DARC policy	No	99.0%	90.0%	94.50%	1.5

6.5 Cost-Success Tradeoff

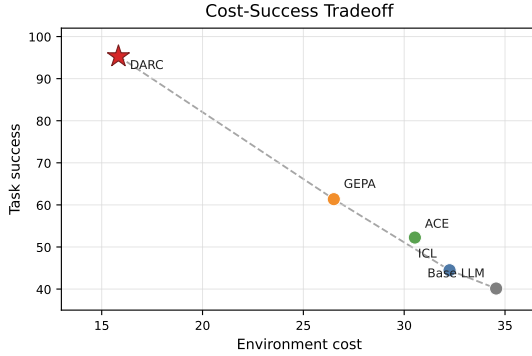


Figure 4: Cost-success tradeoff on AppWorld. The red star marks the distilled DARC policy; points farther right use more environment steps.

Figure 4 visualizes the candidate AppWorld policies. A policy that stacks many interventions can improve coverage but increases mean environment steps. A policy that is too short saves cost but misses recoverable tasks. The selected DARC policy lies on the observed high-success, lower-cost frontier for this candidate set.

6.6 Cost and Speed Efficiency

Table 10: Cost and speed efficiency on ALFWorld with DeepSeek-V4-Flash. DARC improves success per 100 environment steps by reducing invalid actions and shortening trajectories.

Method	Env Steps↓	Invalid/ep.↓	Sec./ep.↓	Succ./100 steps↑
Base LLM	34.56	1.288	155.4	1.16
ICL	32.25	2.453	23.3	1.38
GEPA	26.51	1.489	20.0	2.31
ACE	30.53	1.343	143.8	1.71
DARC	15.83	0.091	10.3	6.02

Table 10 shows the cost side of the ALFWorld recovery policy. Relative to the base agent, DARC reduces average environment steps by 54.2% and invalid actions by 92.9%. It also improves success per 100 steps from 1.16 to 6.02.

Figure 3 shows the budget-level view of the same effect: DARC solves more episodes than each baseline at every maximum environment-step budget. These numbers support the claim that diagnosis-guided recovery can improve both success and cost in an action-validity setting.

7 Conclusion and Limitations

In this work, we introduced DARC, a framework that recasts agent self-correction from indiscriminate context expansion to diagnosis-guided recovery. By profiling task-family failure modes to restrict admissible interventions and distilling a cost-aware policy, DARC improves success across ALF-World, AppWorld, and XBRL Finance while mitigating the overhead and interference of generic playbooks. Future work includes extending our static diagnosis to dynamic, instance-level routing for mixed-mode failures, developing more sample-efficient policy distillation methods, and expanding full-library cascade comparisons to broader domains.

References

- [1] Rishabh Agarwal, Avi Singh, Lei Zhang, Bernd Bohnet, Luis Rosias, Stephanie Chan, Biao Zhang, Ankesh Anand, Zaheer Abbas, Azade Nova, et al. Many-shot in-context learning. *Advances in Neural Information Processing Systems*, 37:76930–76966, 2024.
- [2] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqi, and Mausam. Automix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*, 2023.
- [3] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [4] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce-style optimization for learning from human feedback in llms. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, 2024.
- [5] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.

- [6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. In *arXiv preprint arXiv:2108.07732*, 2021.
- [7] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. *Advances in Neural Information Processing Systems*, 37:12461–12495, 2024.
- [8] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. In *arXiv preprint arXiv:2212.08073*, 2022.
- [9] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 3119–3137, 2024.
- [10] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.
- [11] Lingjiao Chen, Matei Zaharia, and James Zou. Frugal-gpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.
- [13] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations*, 2024.
- [14] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. In *arXiv preprint arXiv:2110.14168*, 2021.
- [15] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.
- [16] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1762–1777, 2023.
- [17] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Paspapat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning*, pages 3929–3938, 2020.
- [18] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.
- [19] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *International Conference on Learning Representations*, 2024.
- [20] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.
- [21] Yihao Hu, Zhihao Wen, Xiujiu Liu, Pan Wang, Xin Zhang, and Wei Wu. Seal: Synergistic co-evolution of agents and learning environments. *arXiv preprint arXiv:2605.24426*, 2026.
- [22] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880, 2021.
- [23] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Few-shot learning with retrieval augmented language models. *arXiv preprint arXiv:2208.03299*, 1 (2):4, 2022.
- [24] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the*

- 2024 *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, 2024.
- [25] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations*, 2024.
- [26] Eyal Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Haggai Bata, Yoav Levine, Kevin Leyton-Brown, et al. MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. In *arXiv preprint arXiv:2205.00445*, 2022.
- [27] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.
- [28] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [29] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *International Conference on Learning Representations*, 2023.
- [30] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. Language models can solve computer tasks. *Advances in Neural Information Processing Systems*, 36:39648–39677, 2023.
- [31] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213, 2022.
- [32] Harrison Lee, Sang Michael Xie Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kelvin Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. RLAI: Scaling reinforcement learning from human feedback with ai feedback. In *arXiv preprint arXiv:2309.00267*, 2023.
- [33] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [34] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for mind exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.
- [35] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. LooGLE: Can long-context language models understand long contexts? *arXiv preprint arXiv:2311.04939*, 2024.
- [36] Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented llms. *arXiv preprint arXiv:2304.08244*, 2023.
- [37] Zeping Li, Hongru Wang, Yiwen Zhao, Guanhua Chen, Yixia Li, Keyang Chen, Yixin Cao, Guannan Ye, Hongfeng Chai, and Zhenfei Yin. Rethinking the role of entropy in optimizing tool-use behaviors for large language model agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27955–27967, 2026.
- [38] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. *arXiv preprint arXiv:2310.10505*, 2023.
- [39] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. In *Transactions on Machine Learning Research*, 2023.
- [40] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.
- [41] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

- [42] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [43] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046, 2024.
- [44] Zhiwei Liu, Weiran Yao, Jianguo Zhang, Liangwei Yang, Zuxin Liu, Juntao Tan, Prafulla K Choubey, Tian Lan, Jason Wu, Huan Wang, et al. Agentlite: A lightweight library for building and advancing task-oriented llm agent system. *arXiv preprint arXiv:2402.15538*, 2024.
- [45] Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutsopoulos, and Georgios Paliouras. Finer: Financial numeric entity recognition for xbrl tagging. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4419–4431, 2022.
- [46] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594, 2023.
- [47] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*, pages 26106–26128. PMLR, 2023.
- [48] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, 2024.
- [49] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- [50] Licheng Pan, Yongqi Tong, Xin Zhang, Xiaolu Zhang, Jun Zhou, and Zhixuan Chu. Understanding and mitigating overrefusal in llms from an unveiling perspective of safety decision boundary. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 21057–21075, 2025.
- [51] Licheng Pan, Haochen Yang, Haoxuan Li, Yunsheng Lu, Yongqi Tong, Yinuo Wang, Shijian Wang, Zhixuan Chu, Lei Shen, Yuan Lu, and Hao Wang. Optimal transport for llm reward modeling from noisy preference. *arXiv preprint arXiv:2605.06036*, 2026.
- [52] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. ART: Automatic multi-step reasoning and tool-use for large language models. In *arXiv preprint arXiv:2303.09014*, 2023.
- [53] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [54] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- [55] Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Jiadai Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, et al. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, volume 2025, pages 79791–79821, 2025.
- [56] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The twelfth international conference on learning representations*, 2023.
- [57] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- [58] Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026. URL <https://qwen.ai/blog?id=qwen3.6-27b>.
- [59] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.

- [60] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Tool-former: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [61] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [62] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.
- [63] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, pages 31210–31227, 2023.
- [64] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [65] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020.
- [66] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Austin Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023.
- [67] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.
- [68] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023.
- [69] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. *arXiv preprint arXiv:2408.02442*, 2024.
- [70] Yongqi Tong, Yifan Wang, Dawei Li, Sizhe Wang, Zi Lin, Simeng Han, and Jingbo Shang. Eliminating reasoning via inferring with planning: A new framework to guide llms’ non-linear thinking. *arXiv preprint arXiv:2310.12342*, 2023.
- [71] Yongqi Tong, Dawei Li, Sizhe Wang, Yujia Wang, Fei Teng, and Jingbo Shang. Can llms learn from previous mistakes? investigating llms’ errors to boost for reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3065–3080, 2024.
- [72] Yongqi Tong, Sizhe Wang, Dawei Li, Yifan Wang, Simeng Han, Zi Lin, Chengsong Huang, Jiaxin Huang, and Jingbo Shang. Optimizing language model’s reasoning abilities with weak supervision. *arXiv preprint arXiv:2405.04086*, 2024.
- [73] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024.
- [74] Dannong Wang, Jaisal Patel, Daochen Zha, Steve Y Yang, and Xiao-Yang Liu. Finlora: Benchmarking lora methods for fine-tuning llms on financial datasets. *arXiv preprint arXiv:2505.19819*, 2025.
- [75] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [76] Pan Wang. REFLEX: Reflective evolution from LLM experience. In *ICML 2026 AI for Science Workshop*, 2026. URL <https://openreview.net/forum?id=xftknNrW7n>.
- [77] Pan Wang, Yihao Hu, Xiu Jin Liu, Jingchu Yang, Hang Wang, and Zhihao Wen. Atlasva: Self-evolving visual skill memory for teacher-free vlm agents. *arXiv preprint arXiv:2605.17933*, 2026.
- [78] Renxi Wang, Haonan Li, Xudong Han, Yixuan Zhang, and Timothy Baldwin. Learning from failure: Integrating negative examples when fine-tuning large language

- models as agents. *arXiv preprint arXiv:2402.11651*, 2024.
- [79] Sizhe Wang, Yongqi Tong, Hengyuan Zhang, Dawei Li, Xin Zhang, and Tianlong Chen. BPO: Towards balanced preference optimization between knowledge breadth and depth in alignment. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8811–8826, 2025.
- [80] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yue Song, Bowen Li, Jaskirat Singh, et al. OpenDevin: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [81] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [82] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [83] Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=hH36JeQZDaO>.
- [84] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- [85] Chunqiu Steven Xia, Yinlin Deng, Sam Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [86] Zhihao Xu, Yongqi Tong, Xin Zhang, Jun Zhou, and Xiting Wang. Understanding conflicts in multi-objective alignment through reward consistency. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 5450–5472, 2026.
- [87] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.
- [88] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [89] Minglai Yang, Ethan Huang, Liang Zhang, Mihai Surdeanu, William Yang Wang, and Liangming Pan. How is llm reasoning distracted by irrelevant context? an analysis using a controlled benchmark. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13340–13358, 2025.
- [90] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [91] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822, 2023.
- [92] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [93] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, volume 35, pages 15476–15488, 2022.
- [94] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=eC4ygDs02R>.
- [95] Li Zhong, Zilong Wang, and Jingbo Shang. Debug like a human: A large language model debugger via verifying runtime execution step-by-step. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 851–870, 2024.
- [96] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023.

[97] Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. Archer: Training language model agents via hierarchical multi-turn rl. *arXiv preprint arXiv:2402.19446*, 2024.

A Theoretical Proofs

A.1 Proof of Monotone Submodularity

For an intervention subset A , a task is covered if at least one intervention in A succeeds on it. Thus $f(A) = \frac{1}{n} \sum_i \mathbf{1}[\exists r \in A : s(x_i, r) = 1]$ is a standard coverage function. Coverage functions are monotone because adding an intervention cannot remove a covered task, and submodular because the marginal gain of adding an intervention decreases as the covered set grows.

A.2 Uniform-Convergence Bound for a Fixed Chain Class

Let Σ_K be a finite class of duplicate-free chains of length at most K , and suppose per-task cost is bounded by $C_{\max}$. For a fixed chain S , Hoeffding’s inequality bounds the deviation of both empirical success and empirical cost. Applying a union bound over Σ_K yields a uniform deviation term of order $O(\sqrt{\log |\Sigma_K|/n})$ for the penalized objective. The empirical maximizer is therefore near the best chain in the fixed class up to twice this deviation. This bound does not replace empirical validation; it only explains why reducing $|\Sigma_K|$ can reduce finite-sample search risk.

B Extended Related Work

Cascades and adaptive computation. DARC is structurally related to LLM cascades and adaptive computation. Frugal-GPT [11] and AutoMix [2] route among language models to reduce cost, while Adaptive-RAG [24] adjusts retrieval complexity. DARC instead routes among recovery interventions inside an agent harness. The base model is fixed; what changes is whether the agent receives an action guard, procedural source, retrieval evidence, or another admissible recovery operation.

Broader evaluation context. The benchmarks in this paper sit within a broader evaluation ecosystem for language-model capability, instruction following, tool use, and interactive autonomy [18, 39, 43, 66, 68, 92]. This ecosystem increasingly shifts from single-turn answer quality toward process observability: intermediate reasoning, external tool calls, state changes, and verifiable task completion [15, 29, 44, 53]. DARC follows this shift but focuses on what to do after failure: it treats failures as signals for selecting recovery interventions rather than as final outcomes alone.

DIAGNOSIS AND RECOVERY POLICY

[diag-0007]

When the low-cost Auto-Knowledge source fails, DARC treats the task family as procedural-recovery dominated. The policy invokes richer context only for unresolved tasks, preserving an explicit failure boundary.

DISTILLED FALLBACK POLICY

[policy-0012]

Selected AppWorld policy: $ak_r1 \rightarrow local_r2 \rightarrow local_r1$. It solves 27/30 tasks with mean 1.5 attempts, successes split as $ak_r1=20$, $local_r2=5$, $local_r1=2$.

AUDITABLE TRACE AND FAILURE BOUNDARY

[trace-0003]

Venmo friend-synchronization task: ak_r1 exhausts its budget, $local_r2$ recovers the workflow, and the policy stops. Trace: ak_r1 : fail $\rightarrow local_r2$: success $\rightarrow stop$.

Figure 5: Example AppWorld recovery trace. The distilled procedural policy invokes additional context only after a cheaper intervention fails.

C Recovery-Harness Mapping

Table 11 summarizes the evaluated task-family mappings. The mapping is frozen before test evaluation and should be interpreted as the recovery-harness instantiation used in this paper, not as a learned universal diagnostic router.

D Representative Diagnosis Records

Table 12 makes the diagnosis interface concrete. The diagnostic LLM receives only development-set trajectory summaries and verifier-visible failure signals, then maps the dominant recurring pattern to one label from the fixed failure-mode vocabulary. These task-family diagnoses and their associated intervention sets are frozen before test evaluation.

E Generated Recovery Trace Example

Figure 5 illustrates the AppWorld procedural recovery harness. The distilled policy first uses a lower-cost procedural source and invokes richer context only when the earlier intervention fails, reducing unnecessary context calls while preserving recovery capacity.

F Factorial Decomposition of the ALFWorld Recovery Harness

Section 6.2 shows that neither privileged information nor action-space restriction alone explains the ALFWorld gain. This appendix reports the underlying 2×2 factorial experiment that separates the two components of the action-validity harness: the *restricted action view* produced by the guard of Eq. 8, and the *recovery prompt* that instructs the agent how to act inside that view.

Protocol. All configurations run on the full ALFWorld valid_unseen split (134 tasks) with DeepSeek-V4-Flash, the full set of admissible actions returned by the environment, a 50-step budget, temperature 0, and the same evaluation script. The two factors are toggled independently: the action view is either the full admissible set or the guard-ranked top-12, and the recovery prompt is either absent or present. We report exact task counts so that paired tests can be reproduced from the released per-task outcomes. Success is reported

Table 11: Frozen task-family to recovery-harness mapping. The diagnostic signal is measured on development tasks; the admissible intervention set is then fixed for test-time policy distillation and deployment.

Task Family	Development-Set Signal	Failure Mode	Admissible Intervention Set
ALFWorld	Invalid / state-incompatible actions	Action validity	Action pruning and admissibility guards
AppWorld	Missing API workflow / multi-step procedure failures	Procedure breadth	Auto-Knowledge, local induction, retrieval fallback
Finance	Exact formula/tag format errors	Format precision	Retrieval few-shot demonstrations and budget selection

Table 12: Representative development-trace diagnosis examples. The trace summaries compress recurring observable failures; no test labels or hidden benchmark state are supplied to the diagnostic LLM.

Benchmark	Observed Failure Trace	LLM Diagnosis
ALFWorld	The agent repeatedly issues actions that are unavailable in the current environment state, making no state progress across consecutive steps.	Action validity
AppWorld	API calls omit a required upstream dependency, such as resolving an entity or resource identifier before invoking the downstream cross-application operation.	Procedure breadth
Finance	The retrieved evidence supports the correct formula or tag, but the response contains prose, extra fields, or a malformed schema that fails exact-match evaluation.	Format precision

as solved task count over 134. Neither factor is effective in isolation; the gain appears only when the recovery prompt is paired with the restricted action view.

Table 13: Factorial decomposition of the action-validity harness on ALFWorld valid_unseen.

Action view	Recovery prompt	Solved	Success $\uparrow$
Full admissible set	Absent	53 / 134	39.55%
Full admissible set	Present	52 / 134	38.81%
Guard-ranked top-12	Absent	58 / 134	43.28%
Guard-ranked top-12	Present	119 / 134	88.81%

The effect is an interaction, not a sum of main effects.

Holding the action view at the full admissible set, adding the recovery prompt changes success by -0.75 pp ($53 \rightarrow 52$ tasks). Holding the recovery prompt absent, restricting the action view to the guard-ranked top-12 changes success by $+3.73$ pp ($53 \rightarrow 58$ tasks). The two single-factor effects therefore account for $+2.98$ pp in total, whereas enabling both factors yields $+49.26$ pp ($53 \rightarrow 119$ tasks). The difference-in-differences interaction term is

$$88.81 - 43.28 - 38.81 + 39.55 = +46.27 \text{ pp}, \quad (9)$$

so essentially the entire effect is attributable to the pairing rather than to either component. This is the empirical form of the Intervention Mismatch argument in Section 1: a recovery signal is only useful when it lands on an interface that can execute it.

Invalid actions explain the mechanism. The three configurations for which we measured action-level statistics show why the prompt is inert without the restricted view. On the full admissible set, the recovery prompt *increases* invalid actions per episode from 1.709 to 2.507 (Table 7):

the prompt tells the agent what the task requires, but many of the resulting commands are not executable in the current state, so the additional guidance is spent on rejected actions. Under the guard-ranked view the same guidance becomes executable and invalid actions fall to 0.575. The recovery prompt does not add knowledge the agent lacked; it becomes actionable only once the action interface is compatible with it.

Auxiliary controls. Two further controls bound the contribution of the guard itself. First, forcing the top-1 guard-ranked command at every step *without any language model* in the loop reaches 40.33%, statistically indistinguishable from the 39.55% base agent. The guard is therefore a high-recall candidate filter rather than a hand-written solver: its top-1 ordering is usually not the correct action, but the correct action is retained within the top-12 while distractors are removed. Second, attaching the same ranked view and recovery prompt to a plain base agent, without the remaining harness components, reaches 82.28%, so most of the effect is carried by the restriction-prompt pairing and 6.53 pp remain attributable to the rest of the harness.

Scope of this decomposition. These are single-run results at temperature 0 on one split and one backbone, so the small single-factor effects (-0.75 pp and $+3.73$ pp, i.e. one and five tasks) should be read as null rather than as measured directions. The joint effect (66 tasks) is far outside this range. We report the decomposition for ALFWorld only; the corresponding factorial for the AppWorld procedural harness and the Finance retrieval harness is not available, and whether the same superadditive structure holds for those failure modes remains an open question.

G Baseline Descriptions

We compare against two groups of methods:

- **Base LLM**: The unaugmented model using a standard ReAct-style execution loop without additional recovery-harness construction.
- **ICL**: Standard prompting with a fixed set of demonstrations sampled or retrieved from the training set.
- **MIPROv2**: A DSPy prompt-optimization method that jointly tunes instructions and demonstrations.
- **GEPA**: A reflective prompt-evolution method that mutates prompts based on execution feedback.
- **ACE**: a broad recovery-playbook baseline constructed from training-set execution feedback.

All applicable offline-adaptation baselines use the same training split and adaptation budget as DARC. We further include a stronger matched baseline that performs validation-selected intervention-chain search without diagnosis-guided restriction (the full-library cascade in Table 14); extending it from ALFWorld to the other task families remains future work.

H Isolating the Diagnosis Step

The generic and mismatched conditions in Section 6 vary the intervention set but do not run the same policy search as DARC. We therefore add the strongest matched control: a *full-library cascade* that performs the identical validation-selected policy search as DARC. This control shares the same base outputs, verifier, clean-restart protocol, maximum chain length $L=3$, cost penalty $\lambda=0.02$, and selection splits, but it omits the diagnosis step that first restricts the candidate library. Table 14 reports the result on ALFWorld. Both cascades share the same base outputs, verifier, clean-restart protocol, maximum chain length $L=3$, and cost penalty $\lambda=0.02$, and are selected on `valid_seen` and frozen for `valid_unseen`; the only variable is whether diagnosis restricts the candidate recovery library before search. The diagnosed row re-selects a cascade (`type_saturation` $\rightarrow$ `loop_fallback` $\rightarrow$ `action_pruning`) over the four action-validity harnesses, so its accuracy differs from the fixed single-harness DARC reported in Table 3. Removing diagnosis does not produce a significant accuracy gain: the full-library cascade reaches 98.51% test accuracy versus 99.25% for the diagnosed cascade ($\Delta=+0.75$ pp, 95% CI [0.00, 2.24]; not significant). Diagnosis therefore does not trade accuracy for efficiency—it attains statistically indistinguishable accuracy while searching a $10\times$ smaller policy space (40 vs. 400 candidate policies) and selecting a markedly more stable policy (14 vs. 34 distinct chains under resampling). This empirically supports the finite-sample argument in Section 4.3: restricting $\mathcal{R}_m$ shrinks $|\Sigma_K|$ and the associated search risk at no measured accuracy cost.

Table 14: Isolating the diagnosis step on ALFWorld (DeepSeek-V4-Flash).

Method	Search space	Selection	Test	Distinct chains
DARC (diagnosed)	40	97.14%	99.25%	14
Full-library cascade	400	97.86%	98.51%	34

I Automatic Diagnosis Evaluation

To validate the reliability of the LLM-based failure diagnosis used in DARC, we evaluate its agreement against a verifier-backed reference on a sample of 72 development traces. Table 15 reports the agreement rate and Macro-F1 score. The LLM automatic diagnosis achieves 97.22% agreement with the reference, demonstrating that the failure profiling step reliably identifies the correct dominant failure mode without requiring human labels.

Table 15: Evaluation of the automatic failure diagnosis.

Method	Human Labels	N	Agreement	Macro-F1
Majority	No	72	33.33%	–
Uniform Random	No	72	33.33%	–
LLM Automatic Diagnosis	No	72	97.22%	0.972
Verifier-backed Reference	No	72	100.00%	1.000